



南開大學
Nankai University

《人工智能导论》实验报告

(2025~2026 学年第一学期)

实验名称：基于八数码问题的搜索策略对比

姓 名：李宗杭

学 号：2311451

专 业：软件工程

2025 年 11 月 30 日

目 录

1	问题描述	2
2	问题分析	2
2.1	问题建模	2
2.2	解的存在性	3
3	算法设计与分析	3
3.1	宽度优先搜索 (BFS)	3
3.2	深度优先搜索 (DFS)	5
3.3	启发式搜索算法	7
3.3.1	曼哈顿距离 (Manhattan Distance)	10
3.3.2	欧几里得距离 (Euclidean Distance)	11
3.3.3	错位数 (Misplaced Tiles)	12
3.4	八数码问题最长搜索链的查找	12
3.5	程序可扩展性	14
4	实验及结果分析	15
4.1	不同算法全面比较	15
4.2	程序可扩展性	17
4.3	寻找八数码问题最长搜索链	18
4.4	运行结果	19

1 问题描述

人工智能中的搜索问题是指找到从初始状态到目标状态的路径的问题。这些问题通常涉及到在复杂、有时是动态变化的环境中寻找解决方案。八数码问题是人工智能搜索中的一个典型问题，目前解决八数码的主要方法有深度优先搜索 (DFS)、宽度优先搜索 (BFS)、启发式 A * 算法等等。

本次实验将对这几种算法解决八数码问题的效率进行探讨，并对搜索时间、占用内存、搜索节点数等维度进行比较，通过具体的实验数据分析不同算法策略间的差异。此外，本次实验还将考虑程序可扩展性、最长搜索链等问题。

2 问题分析

2.1 问题建模

八数码涉及一个 3x3 的网格，里面带有 8 个数字的方块和一个空块。每次搜索需要对方块进行移动，来逐步改变棋盘状态以搜索到目标状态。对八数码问题进行建模，棋盘如图2.1所示，展示了八数码问题搜索过程中的初始状态、中间状态和目标状态。如图2.2，可以把八数码问题的求解过程抽象成一个搜索树。

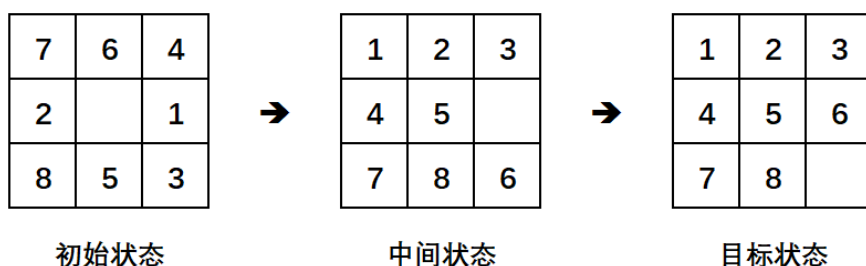


图 2.1: 搜索状态

在算法中需要考虑：

- 状态空间：所有可能的棋盘配置
- 操作：空格向上、下、左、右移动
- 目标测试：当前状态是否与目标状态一致
- 路径代价：移动步数

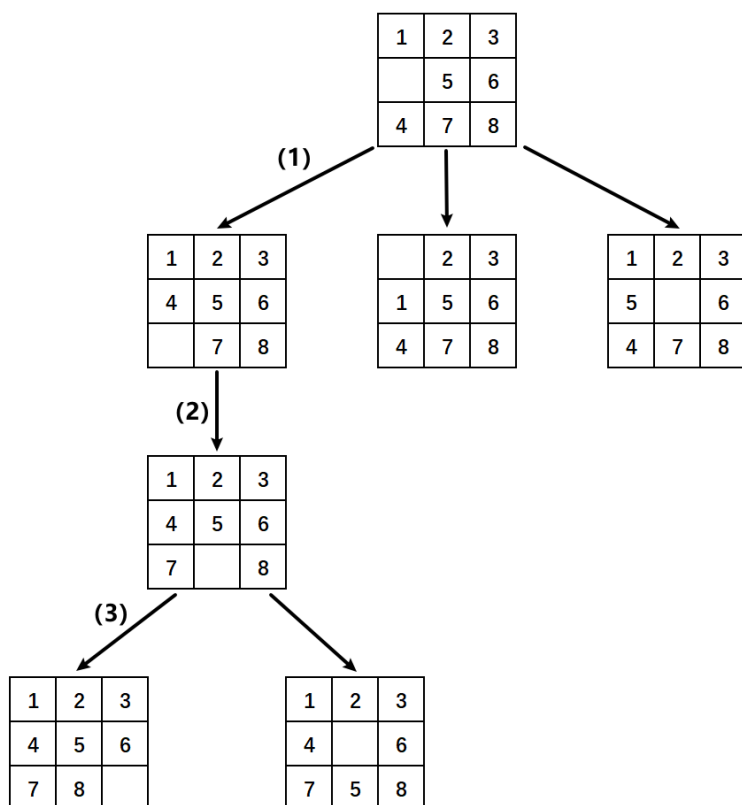


图 2.2: 搜索树

2.2 解的存在性

八数码并不一定都有解，它的解的存在性与初始状态逆序数和空白格的位置有关。在一个排列中，如果一对数的前后位置 and 大小顺序相反，即前面的数大于后面的数，那么它们就成为一个逆序；排列中逆序的总数就称为这个排列的逆序数。

在 N 数码问题中，假设棋盘边长为 N ，即有 $N \times N$ 个格子：若 N 为奇数，一个状态可解当且仅当初始排列与目标排列的逆序数奇偶性相同；若 N 为偶数，一个状态可解当且仅当：初始逆序数奇偶性 + 初始空白格所在行从底部数起的奇偶性 = 目标逆序数奇偶性。

3 算法设计与分析

3.1 宽度优先搜索 (BFS)

宽度优先搜索 (Breadth-First Search, BFS) 是一种用于遍历或搜索树或图的算法。其优先搜索深度浅的节点，每次选择深度最浅的叶节点进行拓展，如果深度相同则按照事先约定的从深度最浅的几个节点中选择一个。在问题有解的情况下，宽度优先搜索一定可以找到最优解。

宽度优先搜索解决八数码问题的步骤如下：

1. 建立 open 表和 close 表，用于存储未访问的状态和已经访问的状态。将初始状态加入到 open 表中，开始搜索。
2. 判断 open 表是否为空，如果为空则失败。如果不为空，取出 open 表中第一个状态，并加入到 close 表中。

3. 判断该状态是否为目标状态，如果是则成功。
4. 对于该节点，拓展它的子节点加入到 open 表。
5. 重复第 2、3、4 步，直到返回失败或者成功。

BFS 代码实现如下：

使用 tracemalloc 记录内存占用情况（仅记录峰值），使用 time.perf_counter() 实现高精度时间记录，会记录生成的节点数和探索的节点数，若搜索成功，则会记录路径长度和完整的路径信息。另外代码中在扩展结点时避免扩展当前节点的父节点，避免进入无限循环（不走来时路）。

```

1  def bfs(self, visualize=False):
2      """
3      宽度优先搜索 (BFS)
4      :param visualize: 是否动态显示搜索过程
5      :return: (成功标志, 路径, 统计信息)
6      """
7      tracemalloc.start() # 启动内存跟踪
8      start_time = time.perf_counter() # 记录开始时间
9
10     queue = deque([self.initial_state]) # 队列：存储待探索状态
11
12     # 初始化统计信息
13     self.stats = SearchStatistics()
14     self.stats.nodes_generated = 1 # 初始状态已生成
15
16     while queue: # 当队列不为空时继续搜索
17         current_state = queue.popleft() # 弹出队列头部状态
18         self.stats.nodes_explored += 1
19
20         if visualize and self.stats.nodes_explored % 100 == 0:
21             print(f"\r探索节点数: {self.stats.nodes_explored}, 队列长度:
22                 ↳ {len(queue)}", end="")
23
24         # 检查是否达到目标
25         if current_state.is_goal(self.goal_state):
26             # 记录成功信息
27             self.stats.success = True
28             self.stats.time_elapsed = time.perf_counter() - start_time
29             self.stats.path_length = current_state.depth
30
31             # 记录内存使用情况
32             _, peak = tracemalloc.get_traced_memory()
33             tracemalloc.stop()

```

```
33         self.stats.memory_used = peak / 1024 / 1024 # 转换为 MB
34
35         # 获取路径
36         path = current_state.get_path()
37         return True, path, self.stats
38
39         # 扩展邻接状态
40         for neighbor in current_state.get_neighbors():
41             if neighbor != current_state.parent if current_state.parent else
42                 ↪ True: # 避免回到父状态
43                 queue.append(neighbor)
44                 self.stats.nodes_generated += 1
45
46         # 未找到解
47         self.stats.time_elapsed = time.perf_counter() - start_time
48         _, peak = tracemalloc.get_traced_memory()
49         tracemalloc.stop()
50         self.stats.memory_used = peak / 1024 / 1024
51         return False, [], self.stats
```

3.2 深度优先搜索 (DFS)

深度优先搜索算法 (Depth-First-Search, DFS) 也是一种用于遍历或搜索树或图的算法。与 BFS 不同, 它每次选择一个深度最深的节点进行扩展, 如果有相同深度的多个节点, 则按照事先规定选择, 尽可能深的搜索树的分支。如果该节点没有子节点了, 就回溯到除了该节点外最深的节点进行扩展, 直到找到问题的答案, 或者再也没有节点可以拓展, 则表示问题无解。深度优先不一定能找到最优解, 而是找到可行的解。

宽度优先搜索解决八数码问题的步骤如下:

1. 建立 open 表和 close 表, 用于存储未访问的状态和已经访问的状态。设置深度限制 max_depth。将初始状态加入到 open 表中, 开始搜索。
2. 判断 open 表是否为空, 如果为空则失败。如果不为空, 取出 open 表中第一个状态, 并加入到 close 表中。
3. 判断该状态是否为目标状态, 如果是则成功。
4. 若达到最大深度限制 max_depth, 则跳过扩展, 继续下一轮循环。
5. 对于该节点, 拓展它的子节点加入到 open 表。
6. 重复第 2、3、4、5 步, 直到返回失败或者成功。

DFS 代码实现如下:

大部分与 BFS 相同, 仅其中 open 表改为栈实现, 并多了一个最大深度判断。

```
1 def dfs(self, max_depth=50, visualize=False):
2     """
3     深度优先搜索 (DFS)
4     :param max_depth: 最大深度限制
5     :param visualize: 是否动态显示搜索过程
6     :return: (成功标志, 路径, 统计信息)
7     """
8     tracemalloc.start()
9     start_time = time.perf_counter()
10
11     stack = [self.initial_state] # bfs 使用队列, dfs 使用栈
12
13     self.stats = SearchStatistics()
14     self.stats.nodes_generated = 1
15
16     while stack:
17         current_state = stack.pop()
18         self.stats.nodes_explored += 1
19
20         if visualize and self.stats.nodes_explored % 100 == 0:
21             print(f"\r探索节点数: {self.stats.nodes_explored}, 栈深度:
22                 ↳ {len(stack)}", end="")
23
24         # 检查是否达到目标
25         if current_state.is_goal(self.goal_state):
26             self.stats.success = True
27             self.stats.time_elapsed = time.perf_counter() - start_time
28             self.stats.path_length = current_state.depth
29
30             _, peak = tracemalloc.get_traced_memory()
31             tracemalloc.stop()
32             self.stats.memory_used = peak / 1024 / 1024
33
34             path = current_state.get_path()
35             return True, path, self.stats
36
37         # 深度限制
38         if current_state.depth >= max_depth:
39             continue
40
41         # 扩展邻接状态
```

```

41     neighbors = current_state.get_neighbors()
42     for neighbor in reversed(neighbors):
43         if neighbor != current_state.parent if current_state.parent else
         ↪ True: # 避免回到父状态
44             stack.append(neighbor)
45             self.stats.nodes_generated += 1
46
47     # 未找到解
48     self.stats.time_elapsed = time.perf_counter() - start_time
49     _, peak = tracemalloc.get_traced_memory()
50     tracemalloc.stop()
51     self.stats.memory_used = peak / 1024 / 1024
52     print("\n未找到解!")
53     return False, [], self.stats

```

3.3 启发式搜索算法

前面所说的 BFS 和 DFS 又称为盲目搜索算法，因其搜索范围比较大，最坏情况下会遍历所有可能的状态，所以效率比较低。如果在搜索过程中引入启发信息，减小搜索范围，就可以加快搜索过程，这种方式称为启发式搜索。

启发式搜索算法是一种在搜索过程中使用启发式方法来优化搜索效率的算法。它们广泛应用于问题解决中，尤其是在问题空间庞大、无法穷举所有可能性的情况下。这类算法的核心在于使用一个启发式函数（也称为评估函数）来估计从当前状态到目标状态的最佳路径。评估函数形如 $f(n) = g(n) + h(n)$ 。其中 n 是待评价的节点， $g(n)$ 是初始节点 s 到节点 n 的最佳路径耗散值的估计值， $h(n)$ 是节点 n 到目标节点 t 的最佳路径耗散值的估计值，也就是启发函数。所以 $f(n)$ 为初始节点 s 到目标节点 t 的最佳路径耗散值的估计值。

在八数码问题中， $g(n)$ 就是为该节点的深度值，而启发函数可以根据问题定义给出不同的函数。如何定义好的启发函数成为使用评估函数的关键所在。

启发式搜索的步骤如下：

1. 建立 open 表和 close 表，用于存储未访问的状态和已经访问的状态。设置深度限制 max_depth 。将初始状态加入到 open 表中，开始搜索。
2. 判断 open 表是否为空，如果为空则失败。如果不为空，取出 open 表中第一个状态，并加入到 close 表中。
3. 判断该状态是否为目标状态，如果是则成功。
4. 对于该节点，拓展它的子节点并计算每个节点的 f 值。
 - (a) 如果该子节点不在 open 表也不在 close 表，则直接加入到 open 表。
 - (b) 如果在 open 表，且 f 值优于 open 中记录的对应子节点，则将 open 中的子节点替换成该子节点。
 - (c) 如果在在 closed 表中，且估值函数值优于 closed 表中对应的子节点，则将 closed 表中对应的子节点移除，并将该子节点重新加入 open 中。

5. 按 f 从小到大排序 open 表。

6. 重复上述步骤。

启发式搜索代码实现如下：

依旧大部分与 BFS 的代码一致，这里的 open 表由优先队列实现，保证每次取出 f 最小的状态。其中 heuristic_func 为启发函数。

```

1  def a_star(self, heuristic_func, visualize=False):
2      """
3      A* 启发式搜索
4      :param heuristic_func: 启发式函数
5      :param visualize: 是否动态显示搜索过程
6      :return: (成功标志, 路径, 统计信息)
7      """
8      tracemalloc.start()
9      start_time = time.perf_counter()
10
11     # 优先队列: (f 值, 计数器, 状态)
12     counter = 0 # 解决 f 值相同的状态排序问题 (避免 heapq 比较状态对象)
13     h = heuristic_func(self.initial_state, self.goal_positions, self.goal_state)
14     pq = [(h, counter, self.initial_state)]
15     open = {self.initial_state._tuple: (counter, self.initial_state.depth)} # 状态 -> (计数器, g 值)
16     open_count = {self.initial_state._tuple: 1} # 状态 -> 出现次数
17     closed = {} # 状态 -> g 值
18
19     self.stats = SearchStatistics()
20     self.stats.nodes_generated = 1
21
22     while pq:
23         f, count, current_state = heappop(pq)
24         latest, _ = open[current_state._tuple]
25         open_count[current_state._tuple] -= 1
26         if open_count[current_state._tuple] == 0:
27             del open_count[current_state._tuple]
28             del open[current_state._tuple]
29         if count < latest:
30             continue # 跳过过时的状态
31         self.stats.nodes_explored += 1
32
33         if visualize and self.stats.nodes_explored % 100 == 0:
34             print(f"\r探索节点数: {self.stats.nodes_explored}, 队列长度: {len(pq)}, f 值: {f:.2f}", end="")

```

```
35
36     # 检查是否达到目标
37     if current_state.is_goal(self.goal_state):
38         self.stats.success = True
39         self.stats.time_elapsed = time.perf_counter() - start_time
40         self.stats.path_length = current_state.depth
41
42         _, peak = tracemalloc.get_traced_memory()
43         tracemalloc.stop()
44         self.stats.memory_used = peak / 1024 / 1024
45
46         path = current_state.get_path()
47         return True, path, self.stats
48
49     closed[current_state._tuple] = current_state.depth
50
51     # 扩展邻接状态
52     for neighbor in current_state.get_neighbors():
53         state_tuple = neighbor._tuple
54         g = neighbor.depth
55         h = heuristic_func(neighbor, self.goal_positions, self.goal_state)
56         f = g + h
57         counter += 1
58
59         if state_tuple == current_state.parent._tuple if
60         ↪ current_state.parent else False:
61             continue # 避免回到父状态
62
63         # 不在 open 表也不在 closed 表, 直接加入 open 表
64         if state_tuple not in open and state_tuple not in closed:
65             heappush(pq, (f, counter, neighbor))
66             open[state_tuple] = (counter, g)
67             open_count[state_tuple] = 1
68             self.stats.nodes_generated += 1
69         # 如果已在 open 中, 检查是否找到更好的路径
70         elif state_tuple in open:
71             _, existing_g = open[state_tuple]
72             if g < existing_g:
73                 heappush(pq, (f, counter, neighbor))
74                 open[state_tuple] = (counter, g)
75                 open_count[state_tuple] += 1
76                 self.stats.nodes_generated += 1
```

```

76         # 如果已在 closed 中, 检查是否找到更好的路径
77         elif state_tuple in closed:
78             existing_g = closed[state_tuple]
79             if g < existing_g:
80                 del closed[state_tuple]
81                 heappush(pq, (f, counter, neighbor))
82                 open[state_tuple] = (counter, g)
83                 if state_tuple in open_count:
84                     open_count[state_tuple] += 1
85                 else:
86                     open_count[state_tuple] = 1
87                 self.stats.nodes_generated += 1
88
89         # 未找到解
90         self.stats.time_elapsed = time.perf_counter() - start_time
91         _, peak = tracemalloc.get_traced_memory()
92         tracemalloc.stop()
93         self.stats.memory_used = peak / 1024 / 1024
94         return False, [], self.stats

```

上述算法可以称作 A 算法, 是基本的启发式搜索, 如果不对其中的 $h(n)$ 做任何约束的话, 其不一定能找到最优解。但如果启发函数满足 $h(n) \leq h^*(n)$, 则可以证明如果问题有解时, 算法一定可以得到耗散值最小的结果, 也就是最佳的解 (可采纳性)。这样的启发式算法被称为 A* 算法。其中 $h^*(n)$ 是从状态 n 到目标状态的实际最短路径长度。

在八数码问题中, $h^*(n)$ 可以用曼哈顿距离表示。因为在八数码中, 每一方块每一次只能移动一格, 而它最少需要移动它到目标方块位置的曼哈顿距离才能达到目标状态。因此曼哈顿距离提供了到达目标状态所需步数的下界估计。

据此, 可以给出以下三种可采纳的启发函数:

3.3.1 曼哈顿距离 (Manhattan Distance)

曼哈顿距离是指在格点上, 从一个点到另一个点的路径长度, 只允许水平或垂直移动。

$$h(n) = \sum (|x_n - x_t| + |y_n - y_t|)$$

在八数码问题中, 每一方块每一次只能移动一格, 而且每次移动只能减少该方块的曼哈顿距离最多一单位, 曼哈顿距离提供了到达目标状态所需步数的下界估计。而它最少需要移动它到目标方块位置的曼哈顿距离才能达到目标状态, 因此它绝对不会夸大损失, 满足 $h(n) \leq h^*(n)$ 的条件, 所以基于曼哈顿距离的启发函数满足条件, 是可采纳的启发函数。

曼哈顿距离代码实现如下:

```

1 def manhattan_distance(state:PuzzleState, goal_positions, goal_state=None):
2     """

```

```

3      曼哈顿距离启发式函数
4      计算每个数字到其目标位置的曼哈顿距离之和
5      :param state: 当前状态
6      :param goal_positions: 目标位置字典, 数字 -> (行, 列)
7      :param goal_state: 目标状态
8      """
9      distance = 0
10     size = state.size
11
12     # 计算曼哈顿距离
13     for i in range(size):
14         for j in range(size):
15             if state.board[i][j] != 0:
16                 value = state.board[i][j]
17                 goal_i, goal_j = goal_positions[value]
18                 distance += abs(i - goal_i) + abs(j - goal_j)
19
20     return distance

```

3.3.2 欧几里得距离 (Euclidean Distance)

欧几里得距离是两点之间的“直线”距离。

$$h(n) = \sum \sqrt{(x_n - x_t)^2 + (y_n - y_t)^2}$$

由三角形的两条边加起来大于第三边可知, 当在八数码问题求解时, 某个方块到目标方块的欧几里得距离小于等于曼哈顿距离, 而基于曼哈顿距离的启发函数是满足条件的, 因此 $h_{euclidean}(n) \leq h_{manhattan}(n) \leq h^*(n)$ 成立, 所以基于欧几里得距离的启发函数也是可采纳的启发函数。

欧几里得距离代码实现如下:

```

1  def euclidean_distance(state:PuzzleState, goal_positions, goal_state=None):
2      """
3      欧几里得距离启发式函数
4      计算每个数字到其目标位置的欧几里得距离之和
5      :param state: 当前状态
6      :param goal_positions: 目标位置字典, 数字 -> (行, 列)
7      :param goal_state: 目标状态
8      """
9      distance = 0
10     size = state.size
11
12     # 计算欧几里得距离
13     for i in range(size):

```

```

14         for j in range(size):
15             if state.board[i][j] != 0:
16                 value = state.board[i][j]
17                 goal_i, goal_j = goal_positions[value]
18                 distance += math.sqrt((i - goal_i)**2 + (j - goal_j)**2)
19
20     return distance

```

3.3.3 错位数 (Misplaced Tiles)

$h(n)$ = 不在正确位置的数字个数

当某一方块不在该有的位置时，它至少需要移动一次才能到达目标的位置。而某不在原有位置的方块距离目标位置的曼哈顿距离 $d \geq 1$ ，因此基于不在位的方块数的启发函数满足条件，是可采纳的启发函数。

错位数计算方式如下：

```

1  def misplaced_tiles(state:PuzzleState, goal_positions, goal_state=None):
2      """
3      错位数启发式函数
4      计算不在正确位置的数字个数
5      :param state: 当前状态
6      :param goal_positions: 目标位置字典，数字 -> (行，列)
7      :param goal_state: 目标状态
8      """
9      count = 0
10     size = state.size
11     for i in range(size):
12         for j in range(size):
13             if state.board[i][j] != 0 and state.board[i][j] != goal_state[i][j]:
14                 count += 1
15     return count

```

3.4 八数码问题最长搜索链的查找

本实验采用逆向宽度优先搜索 (Reverse BFS) 策略来实现八数码问题最长搜索链的查找。考虑到八数码问题的全排列中约有 181,440 个可解状态，若采用正向思维对每一随机状态分别执行 A* 算法，计算成本极高且存在大量冗余路径搜索。

基于状态转移图的无向性，逆向 BFS 算法选择以目标状态为根节点，利用队列结构逐层向外扩展所有邻接状态。由于 BFS 保证了首次访问节点时的路径为最短路径，因此遍历过程中节点的深度直接对应于该状态还原至目标状态的最优步数。通过遍历整个状态空间的连通分量，算法最终收敛于搜索

树的最底层，此时最后被访问的节点即为距离目标状态最远的初始状态，从而高效地在全状态空间中锁定了求解难度最高的初始状态。

算法实现如下：

```
1 def find_longest_search_path():
2     """
3     寻找 8 数码问题的最长搜索链
4     """
5     print("\n" + "=" * 60)
6     print(" 寻找 8 数码问题的最长搜索路径")
7     print("=" * 60)
8     start_time = time.perf_counter()
9
10    # 扁平化处理以提高速度 (使用元组作为哈希键)
11    # 将二维列表转换为一维元组: ((1,2,3),(4,5,6),(7,8,0)) -> (1,2,3,4,5,6,7,8,0)
12    goal_state = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
13    goal_tuple = tuple(num for row in goal_state for num in row)
14
15    # 预计算邻接关系 (索引 0-8 在 3x3 网格中的移动关系)
16    # 0 1 2
17    # 3 4 5
18    # 6 7 8
19    neighbors_map = {
20        0: (1, 3),
21        1: (0, 2, 4),
22        2: (1, 5),
23        3: (0, 4, 6),
24        4: (1, 3, 5, 7),
25        5: (2, 4, 8),
26        6: (3, 7),
27        7: (4, 6, 8),
28        8: (5, 7),
29    }
30
31    # BFS 初始化
32    queue = deque([(goal_tuple, 0)]) # (状态, 距离)
33    visited = {goal_tuple: 0} # 记录已访问状态及其距离
34
35    max_distance = 0
36    hardest_states = []
37
38    # 开始遍历
39    while queue:
```

```
40     current_state, dist = queue.popleft()
41
42     # 更新最大距离记录
43     if dist > max_distance:
44         max_distance = dist
45         hardest_states = [current_state]
46     elif dist == max_distance:
47         hardest_states.append(current_state)
48
49     # 找到空格 (0) 的位置
50     zero_idx = current_state.index(0)
51
52     # 遍历邻居
53     for neighbor_idx in neighbors_map[zero_idx]:
54         # 创建新状态
55         new_list = list(current_state)
56         new_list[zero_idx], new_list[neighbor_idx] = (
57             new_list[neighbor_idx],
58             new_list[zero_idx],
59         )
60         new_state = tuple(new_list)
61
62         if new_state not in visited:
63             visited[new_state] = dist + 1
64             queue.append((new_state, dist + 1))
65
66     duration = time.perf_counter() - start_time
67     print(f" 遍历完成!")
68     print(f" 耗时: {duration:.4f} 秒")
69     print(f" 总计可解状态数: {len(visited)}")
70     print(f" 最长路径长度: {max_distance}")
71     print(f" 最难解状态数量: {len(hardest_states)}")
```

3.5 程序可扩展性

所有算法中未直接指定状态的大小，而是通过参数 `size` 动态指定，无论是初始状态还是目标状态，亦或是可解性判断中对网格边长的取值，全都是通过 `size` 动态指定，因此只需在调用函数时传入 `size` 值便可实现自定义 `N` 数码问题。对于 15 数码问题，只需将 `size` 设为 4，后续会进行具体演示。

4 实验及结果分析

4.1 不同算法全面比较

通过随机生成初始状态并进行求解，最终提取出 13 种不同难度（以最优解路径长度递增为标准）的八数码问题初始状态，采用 6 种算法进行对比测试：宽度优先搜索（BFS）、深度优先搜索（DFS）、基于曼哈顿距离的 A 算法（A (Manh)）、基于欧几里得距离的 A 算法（A (Eucl)）、基于错位数的 A 算法（A (Mispl)），并引入随机选择策略作为盲目搜索的极端对照。为降低硬件波动对结果的影响，每种算法在每个测试案例中均运行 5 次，取搜索节点数、运行时间、内存占用的平均值作为最终结果，测试数据如表 1、表 2、表 3 所示。

测试结果如下：

案例	BFS	DFS	A* (Manh)	A* (Eucl)	A* (Mispl)	Random
1	1	1	1	1	1	1
2	7	5679	3	3	3	44207
3	25	13168	5	5	5	80005.8
4	108	12149	7	7	7	70999.2
5	371	13651	16	16	25	95647.2
6	771	35898	26	26	45	67445.6
7	2129	26534	24	26	112	100000
8	4447	4048	25	28	173	81129
9	21017	238756	56	80	584	84649
10	55083	735864	71	115	1095	100000
11	176070	389700	664	832	3574	100000
12	477317	952768	1017	1444	8163	100000
13	1433813	2617559	2311	3237	18409	97147.6

表 1: 各算法在不同难度状态下的搜索节点数

案例	BFS	DFS	A* (Manh)	A* (Eucl)	A* (Mispl)	Random
1	0.0637	0.0016	0.0317	0.0099	0.0060	0.0024
2	0.1888	85.6085	0.0856	0.0937	0.0707	1327.5790
3	0.9272	287.0255	0.2220	0.2723	0.1959	3828.1722
4	2.6189	182.7961	0.2393	0.2712	0.2023	2147.0911
5	22.8435	204.6889	0.5402	0.7668	0.7839	2920.9674
6	21.1863	531.8291	0.8741	1.0327	1.4525	2036.9692
7	59.4701	391.4129	0.8085	1.0674	3.8399	3029.9621
8	128.6385	59.8008	0.8302	1.1588	6.1277	2464.2136
9	647.2230	3544.7096	1.9329	3.5731	23.0541	2569.3641
10	1836.6094	11297.2415	2.6180	5.3681	47.2034	3202.2561
11	11939.9656	12918.0353	66.8144	96.4156	356.7539	7023.5939
12	36281.0490	31690.6828	92.8260	181.4102	855.0082	6394.0590
13	130443.5757	106371.3279	272.4027	536.5886	2264.9748	7627.7378

表 2: 各算法在不同难度状态下的运行时间（单位：ms）

案例	BFS	DFS	A* (Manh)	A* (Eucl)	A* (Mispl)	Random
1	0.0009	0.0002	0.0005	0.0005	0.0005	0.0002
2	0.0065	0.0868	0.0028	0.0027	0.0026	33.6049
3	0.0195	0.0628	0.0046	0.0045	0.0044	60.8494
4	0.0939	0.0638	0.0060	0.0059	0.0058	53.9395
5	0.3590	0.0638	0.0147	0.0148	0.0236	72.7643
6	0.8057	0.0639	0.0248	0.0250	0.0464	51.2209
7	2.5227	0.0638	0.0222	0.0241	0.1130	76.0571
8	5.6037	0.0623	0.0233	0.0267	0.1860	61.6599
9	26.7736	0.0647	0.0528	0.0744	0.6107	64.3452
10	69.2803	0.0645	0.0663	0.1097	1.2537	76.0810
11	219.8487	0.0645	0.7199	0.9327	4.6180	76.0570
12	604.8121	0.0645	1.1346	1.7868	10.3377	76.0569
13	1816.5239	0.0654	2.7620	4.0531	22.1205	73.8811

表 3: 各算法在不同难度状态下的内存占用 (单位: MB)

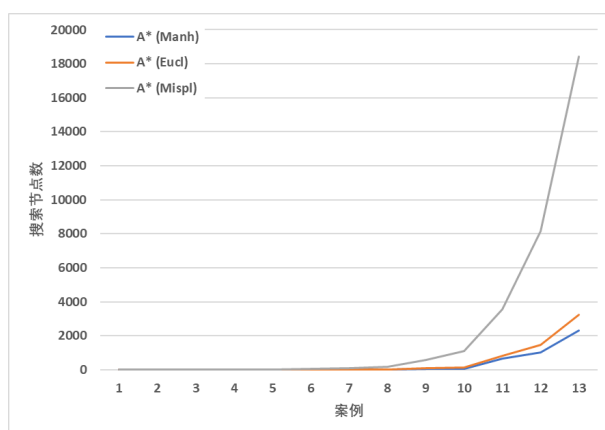


图 4.3: 各 A* 算法在不同难度状态下的搜索节点数

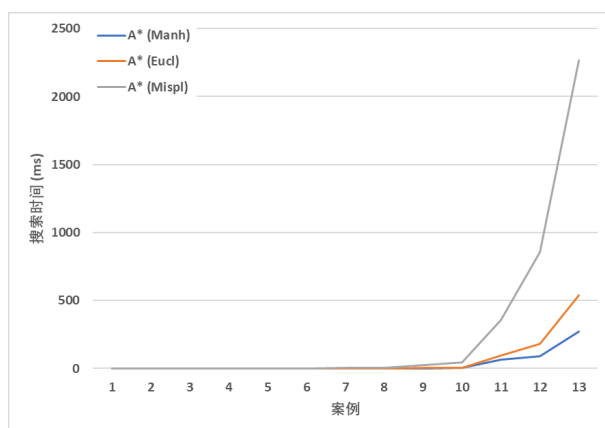


图 4.4: 各 A* 算法在不同难度状态下的运行时间

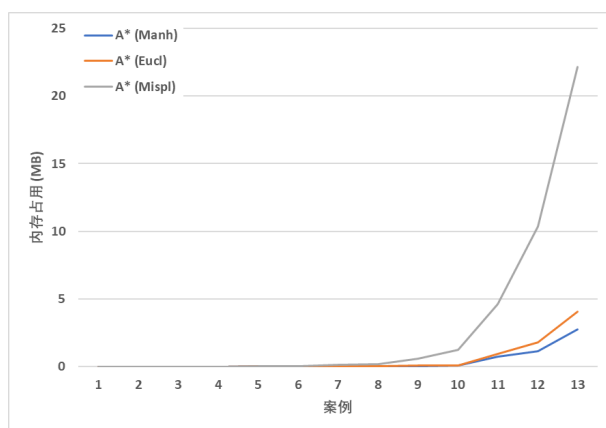


图 4.5: 各 A* 算法在不同难度状态下的内存占用

对比 12 个不同难度的案例，首先分析盲目搜索策略的表现。宽度优先搜索虽然保证能找到最短路径，但其“状态空间爆炸”的问题在测试结果中十分明显。在简单的案例中 BFS 尚能应付，但随着难度增加，各项指标急剧上升。特别是在最难的第 13 个案例中，BFS 不得不搜索超过 143 万个节点，耗时约 130 秒，且内存占用高达 1.8GB，险些导致内存溢出。相比之下，深度优先搜索（DFS）展现了极佳的内存控制能力，无论题目多难，其内存占用始终稳定在 0.06MB 左右，这是因为其只存储当前路径节点。然而，DFS 的搜索时间波动极大，且无法保证找到最优解，容易陷入深层分支或绕远路，因此并不适合求解此类最短路径问题。

实验数据有力地证明了启发式搜索在性能上的压倒性优势，其效率具体取决于启发函数的质量。在三种启发函数的横向对比中，曼哈顿距离的表现最佳。同样是在最难的第 13 个案例中，基于曼哈顿距离的 A 算法仅搜索了 2311 个节点，耗时仅 0.27 秒，内存占用仅 2.7MB。与 BFS 相比，曼哈顿距离将搜索空间压缩了约 600 倍，速度提升了约 480 倍。欧几里得距离表现次之，而错位数启发函数表现最差，其搜索节点数是曼哈顿距离的近 9 倍。这是因为曼哈顿距离比其他函数更紧致地逼近了真实代价，从而产生了更高效的剪枝效果。此外，由图 4.3 和 4.4 和 4.5，可以看出，各算法资源消耗均成指数型增长，折线皆在难度 9 之后爆炸上升。

关于作为对照的随机决策搜索，实验结果表明其在复杂问题上基本不可行。从第 7 个案例开始，随机搜索经常达到 10 万次迭代上限而失败。对于 8 数码这样拥有数十万个状态的空间，缺乏导向的随机乱撞找到目标的概率极低。

最后，针对接下来要测评的 15 数码问题，基于上述数据可以得出明确结论：盲目搜索是完全不可行的。15 数码的状态空间呈指数级增长，BFS 会在瞬间耗尽计算机内存，而 DFS 则会耗尽时间。唯有基于强启发函数（如曼哈顿距离）的 A* 算法具备解决 15 数码问题的潜力。

4.2 程序可扩展性

在介绍算法设计时，已经说明了由于程序中的状态大小由 size 参数控制，因此程序具有良好的可扩展性。并且在上一节中经过分析得到结果，当 size 变大时，盲目搜索并不适合，因此此处给出一个使用 A* 算法进行求解的 15 数码问题以验证程序可扩展性。

```

随机生成的15数码案例：

初始状态：
1 3 4 7
5 □ 2 6
9 10 8 15
13 12 14 11

===== 搜索统计 =====
探索节点数：105
生成节点数：219
路径长度：20
耗时：22.9569 ms
内存使用：0.1864 MB
搜索结果：成功
=====

```

图 4.6: 程序可扩展性

4.3 寻找八数码问题最长搜索链

使用逆向 BFS 来寻找八数码问题的最长搜索链，仅需遍历所有可能的状态，共 $9!$ 种，且其中有一半是不可解的。从测试结果中可以看到，仅需 0.14 秒便能完成查找，因为逆向 BFS 仅需对所有状态进行一次遍历。但若是采用队友状态进行正向求解的话，即使是采用最快的 A^* 算法，也要数个小时才能完成。

测试中还使用 A^* 算法对找到的初始状态进行测试，可以看到结果是无误的。

```

=====
寻找8数码问题的最长搜索路径
=====
遍历完成！
耗时：0.1408 秒
总计可解状态数：181440
最长路径长度：31
最难解状态数量：2

=====
验证：使用  $A^*$ （曼哈顿距离）求解这个最难状态
=====
最难初始状态：
[6, 4, 7]
[8, 5, 0]
[3, 2, 1]

===== 搜索统计 =====
探索节点数：20291
生成节点数：29757
路径长度：31
耗时：1010.7158 ms
内存使用：23.9083 MB
搜索结果：成功
=====

验证结果：算法求出的路径长度为 31
验证成功

```

图 4.7: 寻找八数码问题最长搜索链

4.4 运行结果

```
=====
```

最佳路径: 24

所有案例的平均结果汇总:

1. 节点数:

案例	BFS	DFS	A* (Manhattan)	A* (Euclidean)	A* (Misplaced)	Random
1	1.0	1.0	1.0	1.0	1.0	1.0
2	7.0	5679.0	3.0	3.0	3.0	60001.6
3	25.0	13168.0	5.0	5.0	5.0	80009.8
4	108.0	12149.0	7.0	7.0	7.0	80005.0
5	371.0	13651.0	16.0	16.0	25.0	100000.0
6	771.0	35898.0	26.0	26.0	45.0	100000.0
7	2129.0	26534.0	24.0	26.0	112.0	72000.4
8	4447.0	4048.0	25.0	28.0	173.0	100000.0
9	21017.0	239756.0	56.0	80.0	584.0	100000.0
10	55083.0	735864.0	71.0	115.0	1095.0	81689.8
11	176070.0	389700.0	664.0	832.0	3574.0	100000.0
12	477317.0	952768.0	1017.0	1444.0	8163.0	81409.6
13	1433813.0	2617559.0	2311.0	3237.0	18409.0	83404.0

2. 时间(ms):

案例	BFS	DFS	A* (Manhattan)	A* (Euclidean)	A* (Misplaced)	Random
1	0.0050	0.0016	0.0082	0.0093	0.0057	0.0023
2	0.1789	85.2660	0.0857	0.0945	0.0678	2113.4444
3	1.0510	399.0921	0.2760	0.3238	0.2249	5476.6131
4	6.0153	414.2497	0.4951	0.5796	0.4922	5711.8568
5	19.0005	459.2688	1.0573	1.3649	1.5670	7181.7396
6	45.6527	1228.7508	1.8594	2.2859	2.9736	7104.3907
7	162.0410	916.7412	1.7775	2.7381	8.6771	5089.6111
8	313.5771	148.6174	1.8699	2.5056	13.5133	7229.6948
9	1502.0485	8286.0564	3.8207	7.5483	55.1658	7127.5470
10	4023.4887	25440.3741	5.2996	10.9673	101.5036	5908.4772
11	13264.9987	13251.8157	66.9042	97.7617	354.4625	7026.6457
12	40913.1135	35761.9302	97.1636	189.8933	899.0637	5601.5714
13	109503.6412	80766.7550	218.8571	349.6536	1652.3882	5111.0262

3. 内存(MB):

案例	BFS	DFS	A* (Manhattan)	A* (Euclidean)	A* (Misplaced)	Random
1	0.0009	0.0002	0.0005	0.0005	0.0005	0.0002
2	0.0062	0.0868	0.0027	0.0027	0.0026	45.6553
3	0.0199	0.0628	0.0046	0.0045	0.0044	60.8517
4	0.0934	0.0638	0.0059	0.0059	0.0058	60.8487
5	0.3411	0.0638	0.0147	0.0148	0.0236	76.0584
6	0.8052	0.0639	0.0248	0.0251	0.0464	76.0582
7	2.5317	0.0638	0.0222	0.0242	0.1130	54.6952
8	5.6037	0.0839	0.0233	0.0268	0.1890	76.0743
9	26.7682	0.0647	0.0528	0.0744	0.6107	76.0577
10	69.2850	0.0645	0.0663	0.1097	1.2537	62.0879
11	219.8481	0.0645	0.7199	0.9327	4.6143	76.0573
12	604.8122	0.0645	1.1346	1.7868	10.3377	61.9120
13	1816.5239	0.0654	2.7620	4.0531	22.1205	63.3950

```
=====
```

图 4.8: 不同算法全面比较

图4.8运行结果与 4.1 节中的表格数据不一致，因为在测得 4.1 中数据后未保存运行结果，此处为又一次运行的结果，用作证明程序可运行。（此时已进行完 4.1 节的表格制作与数据分析，便未重新使用新运行结果）。

```

--- 案例 9 ---

初始状态:
5 1 6
4 □ 8
2 3 7

目标状态:
1 2 3
4 5 6
7 8 □

=====
算法                成功率      节点数      时间(ms)      内存(MB)
=====
BFS                  100.00%     21017.0     1502.0485     26.7682
DFS                  100.00%     238756.0    8286.0564     0.0647
A* (Manhattan)       100.00%      56.0       3.8207        0.0528
A* (Euclidean)       100.00%      80.0       7.5483        0.0744
A* (Misplaced)       100.00%     584.0      55.1658       0.6107
Random                0.00%    100000.0    7127.5470     76.0577
=====
最佳路径:  16

--- 案例 10 ---

初始状态:
5 6 □
4 1 8
2 3 7

目标状态:
1 2 3
4 5 6
7 8 □

=====
算法                成功率      节点数      时间(ms)      内存(MB)
=====
BFS                  100.00%     55083.0     4023.4887     69.2850
DFS                  100.00%    735864.0    25440.3741     0.0645
A* (Manhattan)       100.00%      71.0       5.2996        0.0663
A* (Euclidean)       100.00%     115.0      10.9673        0.1097
A* (Misplaced)       100.00%    1095.0     101.5036       1.2537
Random                20.00%    81689.8     5908.4772     62.0879
=====
最佳路径:  18

```

图 4.9: 部分案例的详细测试信息

图4.9展示了部分单个案例的详细测试信息，可以看到随机搜索的成功率极低。